CENTRAL BOARD OF SECONDARY EDUCATION

JAWAHAR NAVODAYA VIDYALAYA

TARIKHET ALMORA UTTARAKHAND 263663

COMPUTER SCIENCE (083)

PROJECT FILE

On

PARKING LOT MANAGEMENT SYSTEM USING STACK

Session: 2026-27

Submitted By:
Name: _____
Class: XII Science
CBSE Roll No: _____

Submitted To:
Pankaj Bhilingwal
PGT Computer Science
J.N.V Tarikhet Almora

ACKNOWLEDGEMENT

I wish to express my deep sense of gratitude and indebtedness to our learned teacher Pankaj Bhilingwal, PGT Computer Science, J.N.V Tarikhet Almora for his invaluable help, advice and guidance in the preparation of this project file.

I am also greatly indebted to our principal D.S Rawat and school authorities for providing me with the facilities and requisite laboratory conditions for making this project file.

I also extend my thanks to a number of teachers, my classmates and friends who helped me to complete this project file successfully.

Signature of Student

CERTIFICATE

This is to certify that,

S. No	Name	CBSE Roll

Student of Class XII, J.N.V Tarikhet Almora has completed the PROJECT FILE titled “Parking Lot Management System Using Stack” during the academic year 2026-27 towards partial fulfilment of credit for the Computer Science Project evaluation of CBSE and submitted satisfactory report, as compiled in the following pages, under my supervision.

Internal Examiner

Signature

External Examiner

Signature

PREFACE

In the field of Computer Science, the term PROJECT refers to a computerization work that is completely performed by using a computer. Today almost every field is computerized, and the different tasks of every field are done using computers and appropriate computer programs. Such programs and software are developed in the form of Projects.

Learning Python in schools is a part of our syllabus, and according to the syllabus worked out by the CBSE, to which my school is affiliated, I enjoyed doing project work for the session 2026-27. I learned a lot, especially learning by doing was a great experience. The report is a summary of what I observed and learnt in school. My humble thanks go to all those concerned with my project.

I also feel that it is the right time to thank my teacher, who guided me during my project, and my parents, friends and project partners for the constant assistance and cooperation extended by them from time to time.

INDEX

S. No	Topic	Page No
1	About Project	6
2	Syllabus Concepts Used	8
3	Flow Chart	9
4	Project Code	10
5	Output	13
6	Technological Requirements: Hardware & Software	19
7	References	19

About Project

Parking Lot Management System Using Stack

The Parking Lot Management System is a menu-driven Python program that manages cars parked in a narrow parking lane. The lane is closed from one side and has only one gate, so the car which enters last is the first one that can come out. This is the Last In First Out (LIFO) principle, which is exactly how the STACK data structure works.

The program implements a stack using a Python list and performs all the standard stack operations - PUSH, POP and PEEK - along with checks for Overflow and Underflow. The parking data is stored permanently in a binary file using the pickle module, so the record of parked cars is not lost when the program is closed.

Project Overview

Objective:

The objective of this project is to understand and implement the stack data structure through a real-life example. The program helps a parking attendant record parked cars, remove cars, calculate parking fees and see which cars need to be moved to take out a particular car.

Key Features:

1. **Park a Car (PUSH):** Adds a new car with car number, owner name and entry time on the top of the stack. Shows OVERFLOW when the lane is full.
2. **Remove Last Car (POP):** Removes the car nearest to the gate and calculates the parking fee. Shows UNDERFLOW when the lane is empty.
3. **Remove Any Car:** Removes a car from the middle of the lane by moving the cars in front of it to a temporary stack and bringing them back afterwards.
4. **Peek:** Shows the car standing nearest to the gate (top of the stack) without removing it.
5. **Display Lane:** Shows all cars from the gate (TOP) to the wall (bottom) with free space available.
6. **Search a Car:** Finds a car and tells how many cars must be moved to take it out.
7. **Permanent Storage:** Saves the lane in the binary file parking_data.dat after every change.

Technology Stack

Programming Language: Python 3

Modules Used: pickle, os, time

The stack is implemented with a Python list, where append() works as PUSH and pop() works as POP. The pickle module stores the list in a binary file, the os module checks whether the data file exists, and the time module records the entry time and calculates the parking fee (Rs. 20 per started hour).

Files in this Project

File	Purpose
parking_lot.py	Main program (menu and stack operations)
parking_data.dat	Binary file storing the parked cars (created automatically)
flowchart.py	Python program that draws the flowchart

	using graphviz
flowchart.png	Flowchart image
img/	Screenshots of the output

How to Run the Program

1. Make sure Python 3 is installed on the computer.
2. Copy the project folder to the computer.
3. Open parking_lot.py in IDLE and press F5 (or run: python parking_lot.py).
4. Choose options from the menu. The maximum lane size is 5 cars (MAX_SIZE).

Syllabus Concepts Used

This project covers the following topics of the CBSE Class XII Computer Science (083) syllabus:

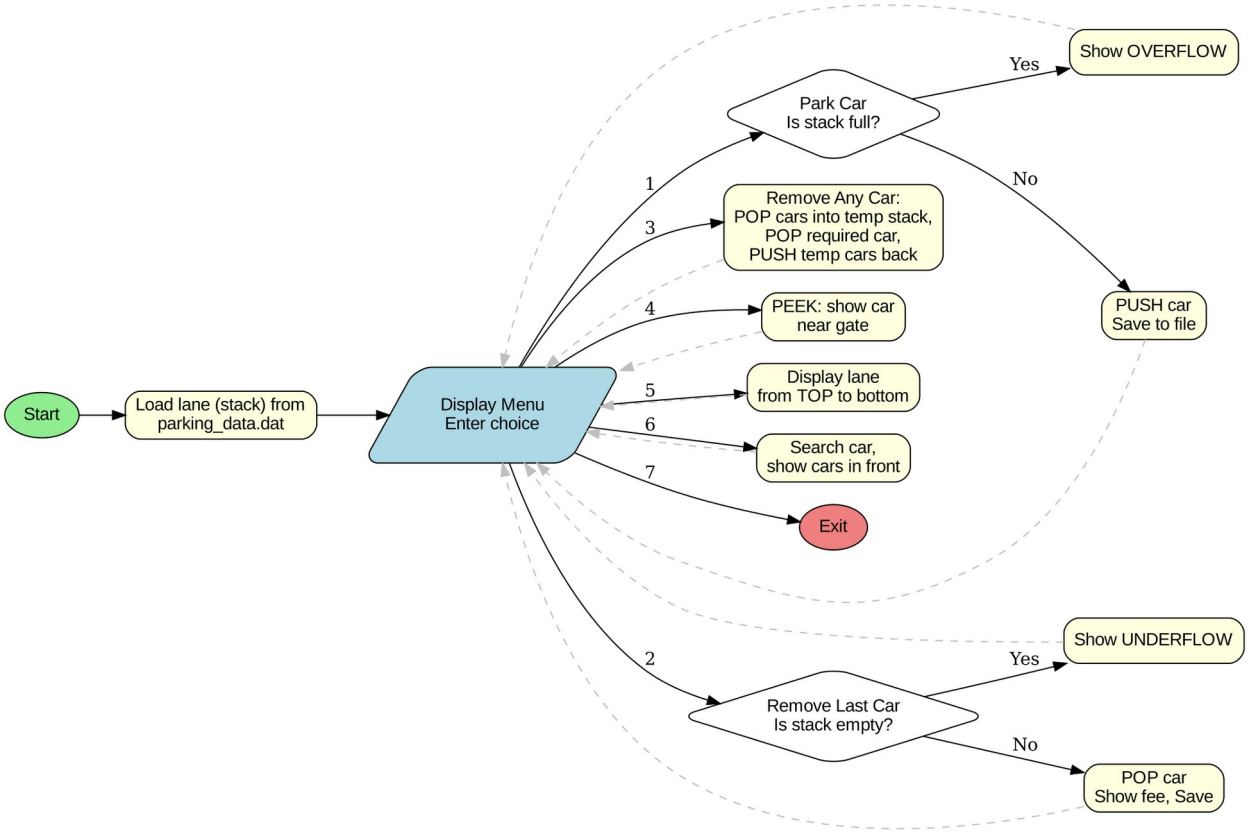
Concept	Where it is used in the project
Stack (LIFO)	Parking lane is a stack: list lane, functions push(), pop(), peek(), is_empty(), is_full()
Overflow / Underflow	park_car() checks is_full(); remove functions check is_empty()
Use of two stacks	remove_any_car() uses a temporary stack temp to move cars out and back
Binary file handling	load_lane() uses pickle.load(), save_lane() uses pickle.dump() with modes 'rb' and 'wb'
User-defined functions	Separate function for every operation; arguments and return values
Lists and nested lists	Each car is a list [car_no, owner, entry_text, entry_seconds]
Built-in modules	os.path.exists(), time.time(), time.strftime()
Loops and conditions	while loop for menu, for loop with range() to display in reverse order

What is a Stack?

A stack is a linear data structure in which insertion and deletion take place only at one end called the TOP. It follows the LIFO (Last In First Out) principle. Common real-life examples are a pile of plates, a stack of books and a narrow parking lane.

PUSH adds an element at the top, POP removes the top element and PEEK only shows the top element. Trying to push into a full stack is called OVERFLOW and trying to pop from an empty stack is called UNDERFLOW.

Flow Chart


Start
Load lane (stack) from parking_data.dat
Display Menu Enter choice
1
Park Car Is stack full?
Yes
Show OVERFLOW
No
PUSH car Save to file
3
Remove Any Car: POP cars into temp stack, POP required car, PUSH temp cars back
4
PEEK: show car near gate
5
Display lane from TOP to bottom
6
Search car, show cars in front
7
Exit
2
Remove Last Car Is stack empty?
Yes
Show UNDERFLOW
No
POP car Show fee, Save

Project Code

```
# Parking Lot Management System using Stack
# The parking lane is closed at one end, so the car that came in LAST
# must go out FIRST (LIFO). This is exactly how a STACK works.

import pickle
import os
import time

FILE_NAME = "parking_data.dat"   # binary file to save the lane
MAX_SIZE = 5                     # maximum cars the lane can hold
RATE_PER_HOUR = 20               # parking charge in Rupees


# ---------------- File handling ----------------

def load_lane():
    # Read the stack (list) from the binary file
    if os.path.exists(FILE_NAME):
        f = open(FILE_NAME, "rb")
        lane = pickle.load(f)
        f.close()
        return lane
    return []


def save_lane(lane):
    # Write the stack (list) to the binary file
    f = open(FILE_NAME, "wb")
    pickle.dump(lane, f)
    f.close()


# ---------------- Stack operations ----------------

def is_empty(lane):
    return len(lane) == 0


def is_full(lane):
    return len(lane) == MAX_SIZE


def push(lane, car):
    lane.append(car)


def pop(lane):
    return lane.pop()


def peek(lane):
    return lane[-1]


# ---------------- Helper functions ----------------

def calculate_fee(entry_time):
    # Every started hour is charged
    seconds = time.time() - entry_time
    hours = int(seconds // 3600) + 1
    return hours * RATE_PER_HOUR


def show_car(car):
    print("Car No:", car[0], "| Owner:", car[1], "| Entry Time:", car[2])


# ---------------- Menu options ----------------
```

```
def park_car(lane):
    if is_full(lane):
        print("OVERFLOW! Parking lane is full. No space for a new car.")
        return
    car_no = input("Enter Car Number: ").upper()
    for car in lane:
        if car[0] == car_no:
            print("This car is already parked.")
            return
    owner = input("Enter Owner Name: ")
    entry_text = time.strftime("%d-%m-%Y %H:%M")
    car = [car_no, owner, entry_text, time.time()]
    push(lane, car)
    save_lane(lane)
    print("Car", car_no, "parked successfully at position", len(lane))


def remove_last_car(lane):
    if is_empty(lane):
        print("UNDERFLOW! Parking lane is empty.")
        return
    car = pop(lane)
    save_lane(lane)
    print("Car", car[0], "has left the parking.")
    print("Parking Fee: Rs.", calculate_fee(car[3]))


def remove_any_car(lane):
    if is_empty(lane):
        print("UNDERFLOW! Parking lane is empty.")
        return
    car_no = input("Enter Car Number to remove: ").upper()

    found = False
    for car in lane:
        if car[0] == car_no:
            found = True
    if not found:
        print("Car", car_no, "is not in the parking lane.")
        return

    temp = []    # temporary stack (waiting area outside the lane)
    # Move cars in front of the required car to the temporary stack
    while peek(lane)[0] != car_no:
        moved = pop(lane)
        push(temp, moved)
        print("Moved car", moved[0], "out to waiting area")

    car = pop(lane)
    print("Car", car[0], "has left the parking.")
    print("Parking Fee: Rs.", calculate_fee(car[3]))

    # Bring the moved cars back into the lane
    while not is_empty(temp):
        moved = pop(temp)
        push(lane, moved)
        print("Car", moved[0], "parked back in the lane")
    save_lane(lane)


def show_gate_car(lane):
    if is_empty(lane):
        print("Parking lane is empty.")
    else:
        print("Car nearest to the gate (TOP of stack):")
        show_car(peek(lane))


def display_lane(lane):
    if is_empty(lane):
```

```
        print("Parking lane is empty.")
        return
    print("------------- GATE -------------")
    for i in range(len(lane) - 1, -1, -1):
        print("Position", i + 1, end=": ")
        show_car(lane[i])
    print("------------ WALL --------------")
    print("Cars parked:", len(lane), "| Free spaces:", MAX_SIZE - len(lane))


def search_car(lane):
    car_no = input("Enter Car Number to search: ").upper()
    for i in range(len(lane)):
        if lane[i][0] == car_no:
            show_car(lane[i])
            cars_in_front = len(lane) - 1 - i
            print("Cars to be moved to take it out:", cars_in_front)
            return
    print("Car", car_no, "is not in the parking lane.")


def main():
    lane = load_lane()
    while True:
        print("\n===== PARKING LOT MANAGEMENT =====")
        print("1. Park a Car (PUSH)")
        print("2. Remove Last Parked Car (POP)")
        print("3. Remove Any Car")
        print("4. Show Car Near Gate (PEEK)")
        print("5. Display Parking Lane")
        print("6. Search a Car")
        print("7. Exit")
        choice = input("Enter your choice: ")

        if choice == "1":
            park_car(lane)
        elif choice == "2":
            remove_last_car(lane)
        elif choice == "3":
            remove_any_car(lane)
        elif choice == "4":
            show_gate_car(lane)
        elif choice == "5":
            display_lane(lane)
        elif choice == "6":
            search_car(lane)
        elif choice == "7":
            print("Thank you! Data saved in", FILE_NAME)
            break
        else:
            print("Invalid choice! Please try again.")


main()
```

Output

Output 1: Parking Cars (PUSH)

Python Shell - Parking Cars (PUSH)

```
===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 1
Enter Car Number: UK01A1234
Enter Owner Name: Ramesh Negi
Car UK01A1234 parked successfully at position 1

===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 1
Enter Car Number: UK02B5678
Enter Owner Name: Sunita Rawat
Car UK02B5678 parked successfully at position 2

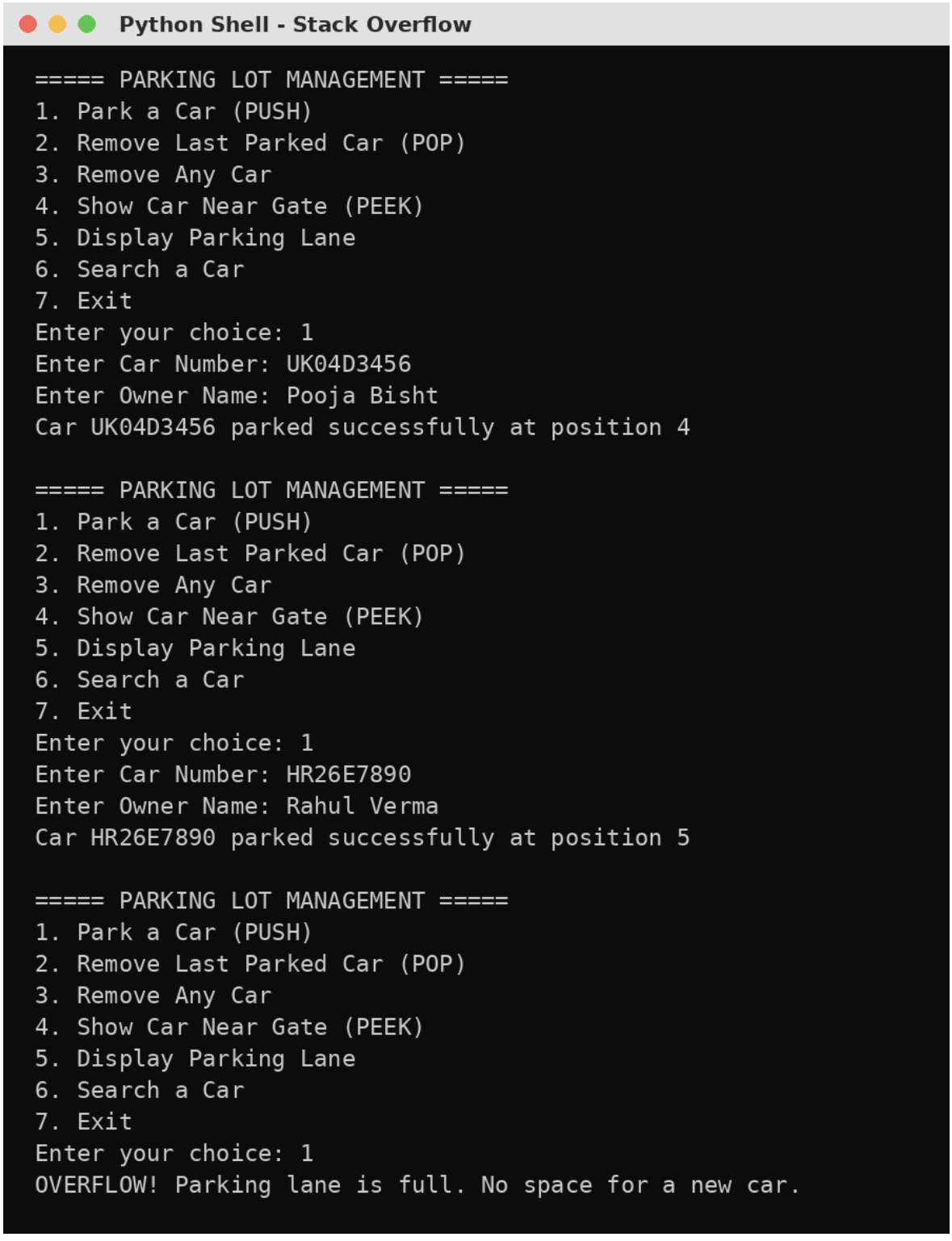
===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 1
Enter Car Number: DL05C9012
Enter Owner Name: Amit Kumar
Car DL05C9012 parked successfully at position 3
```

Output 2: Display Parking Lane

Python Shell - Display Parking Lane

```
===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 5
------------- GATE -------------
Position 3: Car No: DL05C9012 | Owner: Amit Kumar | Entry Time: 07-11-2026 03:23
Position 2: Car No: UK02B5678 | Owner: Sunita Rawat | Entry Time: 07-11-2026 03:23
Position 1: Car No: UK01A1234 | Owner: Ramesh Negi | Entry Time: 07-11-2026 03:23
------------ WALL --------------
Cars parked: 3 | Free spaces: 2
```

Output 3: Stack Overflow



Output 4: Remove a Car from the Middle

Python Shell - Remove a Car from the Middle

```
===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 3
Enter Car Number to remove: UK02B5678
Moved car HR26E7890 out to waiting area
Moved car UK04D3456 out to waiting area
Moved car DL05C9012 out to waiting area
Car UK02B5678 has left the parking.
Parking Fee: Rs. 20
Car DL05C9012 parked back in the lane
Car UK04D3456 parked back in the lane
Car HR26E7890 parked back in the lane

===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 5
------------ GATE ------------
Position 4: Car No: HR26E7890 | Owner: Rahul Verma | Entry Time: 07-11-2026 03:23
Position 3: Car No: UK04D3456 | Owner: Pooja Bisht | Entry Time: 07-11-2026 03:23
Position 2: Car No: DL05C9012 | Owner: Amit Kumar | Entry Time: 07-11-2026 03:23
Position 1: Car No: UK01A1234 | Owner: Ramesh Negi | Entry Time: 07-11-2026 03:23
------------ WALL -------------
Cars parked: 4 | Free spaces: 1
```

Output 5: PEEK and Search

Python Shell - PEEK and Search

```
===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 4
Car nearest to the gate (TOP of stack):
Car No: HR26E7890 | Owner: Rahul Verma | Entry Time: 07-11-2026 03:23

===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 6
Enter Car Number to search: UK01A1234
Car No: UK01A1234 | Owner: Ramesh Negi | Entry Time: 07-11-2026 03:23
Cars to be moved to take it out: 3

===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 6
Enter Car Number to search: MH12Z0000
Car MH12Z0000 is not in the parking lane.
```

Output 6: POP and Exit

Python Shell - POP and Exit

```
===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 2
Car HR26E7890 has left the parking.
Parking Fee: Rs. 20

===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 9
Invalid choice! Please try again.

===== PARKING LOT MANAGEMENT =====
1. Park a Car (PUSH)
2. Remove Last Parked Car (POP)
3. Remove Any Car
4. Show Car Near Gate (PEEK)
5. Display Parking Lane
6. Search a Car
7. Exit
Enter your choice: 7
Thank you! Data saved in parking_data.dat
```

Technological Requirements: Hardware & Software

Software

- Python 3.x (IDLE)
- VS Code / any text editor
- Windows 10/11 or Linux

Hardware

- Laptop / PC with Intel Core i3 or above
- 4 GB RAM or more
- Keyboard and Mouse

References

1. Computer Science with Python - Textbook for Class XII (Sumita Arora)
2. NCERT Computer Science Textbook for Class XII, Chapter: Stack
3. https://www.python.org/
4. https://docs.python.org/3/library/pickle.html
5. https://www.programiz.com/dsa/stack